AGCP: A Deterministic Execution-Layer Governance Control Plane for Autonomous and Programmatic Systems

John M. Willis
Sustainable Future Tech
jwillis@sustainablefuturetech.com
ORCID: 0009-0007-5889-9628

March 29, 2026

Abstract

Autonomous systems, including AI agents and automated infrastructure components, are increasingly capable of proposing and executing operational mutations across enterprise environments. Existing governance mechanisms—such as identity and access management systems, policy engines, and Zero Trust architectures—evaluate permissions and policy compliance but do not define a unified execution-layer model governing how and when system state transitions are committed.

This paper introduces the Artificial Intelligence Governance Control Plane (AGCP), a deterministic control-plane architecture for governing execution at the point of state mutation. AGCP defines a five-stage governance pipeline, an artifact-centric interaction model, explicit commit semantics, and a ledger-derived lifecycle model in which system state is reconstructed from append-only history rather than maintained as mutable state.

The architecture enforces a strict separation between reasoning, governance, and execution, ensuring that all state transitions are mediated by authorization artifacts and lifecycle constraints. Deterministic evaluation and replayability provide reproducibility and auditability, while execution gating ensures that only admissible transitions are committed.

AGCP is grounded in distributed systems principles including deterministic state machines, commit semantics, and log-derived state reconstruction.

Although motivated by the emergence of autonomous agents, AGCP is not limited to AI-based systems. The model applies to any system capable of proposing or executing state transitions, including traditional software systems, infrastructure automation, and distributed services.

AGCP is positioned relative to identity systems, policy engines, Zero Trust architectures, and workflow systems, and is presented as a general execution-layer governance framework for deterministic and verifiable control of system state transitions. AGCP is formulated as a deterministic state-machine control plane in which system state is derived from an ordered log of transitions.

Keywords: governance control plane, execution-layer governance, deterministic systems, state transition governance, policy enforcement, distributed systems, autonomous systems, AI governance, auditability, system-of-record, control plane architecture

Cite as: Willis, J. (2026). *AGCP: A Deterministic Execution-Layer Governance Control Plane for Autonomous and Programmatic Systems*. Sustainable Future Tech, Inc.
https://doi.org/10.5281/zenodo.19323672

1 Introduction

Autonomous systems, including AI agents and automated infrastructure components, are increasingly capable of proposing and executing operational mutations across enterprise environments. These mutations include identity provisioning, infrastructure modification, financial transactions, and policy updates.

The emergence of AGCP is driven in part by the increasing prevalence of autonomous agents capable of proposing and executing system mutations. These systems expose limitations in existing governance mechanisms when operating at machine speed and scale. However, the applicability of AGCP is not limited to AI-based systems. Any system capable of proposing or executing state transitions—including traditional software systems, automation frameworks, and distributed infrastructure—can benefit from deterministic execution-layer governance. AGCP is therefore defined as a general control-plane architecture for governing state transitions, independent of the origin of those transitions.

Existing governance mechanisms—such as identity systems, policy engines, and Zero Trust architectures—evaluate permissions and policy compliance but do not define a unified deterministic execution-layer governance model governing how and when system state may be modified (Sandhu et al., 1996; Rose et al., 2020; Schneider, 2000).

This gap is associated with failure modes including:

- time-of-check/time-of-use inconsistencies (Bishop and Dilger, 1996)
- race conditions between authorization and execution (Lamport, 1978)
- reliance on mutable workflow state
- non-reproducible governance outcomes (Kleppmann, 2017)

A system-level mechanism is therefore required to govern execution at the point of state mutation.

This paper introduces the **Artificial Intelligence Governance Control Plane (AGCP)** as a concrete execution-layer governance system that enforces deterministic, policy-constrained state transitions.

AGCP defines governance as:

> Execution-time enforcement of admissible state transitions through a deterministic control plane.

1.1 Contributions

This paper makes the following contributions:

1. Defines AGCP as a concrete execution-layer governance control plane system
2. Introduces a five-stage deterministic governance pipeline
3. Formalizes commit semantics as an explicit execution boundary
4. Defines a normative artifact model for governance interaction
5. Establishes ledger-derived lifecycle state as a function of append-only history
6. Introduces execution gating based on authorization artifacts and lifecycle constraints
7. Positions AGCP relative to identity and access management (IAM), policy engines (e.g., Open Policy Agent (OPA)), Zero Trust architectures, and workflow systems
8. Clarifies the relationship between AGCP, Practitioner's Blueprint for Secure Artificial Intelligence (PBSAI), Runtime Governance Architecture (RGA), and formal execution semantics

1.2 Paper Organization

Section 2 describes related work. Section 3 defines the problem and control plane concept. Sections 4–9 describe the AGCP system, pipeline, artifact model, commit semantics, and state model. Section 10 defines the interface model. Section 11 describes conformance and determinism requirements. Section 12 compares AGCP to existing systems. Section 13 situates AGCP within the broader architecture. Section 14 provides a condensed formal model and threat model. Section 15 describes performance considerations. Section 16 discusses limitations. Section 17 concludes.

2 Related Work

AGCP is situated within a broader body of work spanning governance architectures, distributed systems, and AI control frameworks. This section positions AGCP relative to prior work, including its own specification lineage and related architectural and formal models.

2.1 AGCP Specification

The AGCP system described in this paper is defined formally in the Artificial Intelligence Governance Control Plane (AGCP) Specification v0.9.0 (Willis, 2026a). The specification provides normative definitions of:

- artifact schemas
- policy evaluation contracts
- lifecycle state transitions
- interface semantics

This paper focuses on the architectural model and execution semantics of AGCP, while the specification defines its concrete implementation requirements and conformance constraints.

2.2 PBSAI Governance Ecosystem

The Practitioner's Blueprint for Secure Artificial Intelligence (PBSAI) Governance Ecosystem (Willis, 2026d) defines a multi-agent reference architecture for securing enterprise AI systems. PBSAI introduces:

- governance domains
- agent coordination structures
- policy sourcing and enforcement layers

Within this framework, AGCP serves as the execution-layer control plane responsible for enforcing governance decisions at the point of state transition. While PBSAI defines the broader ecosystem, AGCP provides the deterministic execution mechanism.

2.3 Formal Execution Semantics

The formal properties of governance control planes, including determinism, invariant preservation, and adversarial assumptions, are defined in (Willis, 2026b). That work introduces:

- formal validation functions
- invariant models
- safety guarantees under adversarial conditions

AGCP can be understood as a concrete system-level realization of these formal semantics, translating abstract guarantees into an executable control-plane architecture.

2.4 Runtime Governance Architecture (RGA)

The Runtime Governance Architecture (RGA) (Willis, 2026c) defines a framework for compiling governance intent into executable artifacts and coordinating governance across distributed systems.

RGA addresses:

- governance compilation
- runtime coordination
- cross-system consistency

AGCP operates within this framework as the deterministic execution layer responsible for enforcing compiled governance decisions.

2.5 Distributed Systems Foundations

AGCP is grounded in established distributed systems principles, including deterministic state-machine execution, ordered event logs, and commit semantics (Lamport, 1978; Ongaro and Ousterhout, 2014; Kleppmann, 2017). These foundations inform:

- ledger-based state derivation
- replayability
- deterministic evaluation

While these concepts are well-established, AGCP applies them to the domain of governance execution, where state transitions are subject to policy, invariants, and authorization constraints.

2.6 Positioning

Taken together, these works define a layered view:

- PBSAI: governance ecosystem
- RGA: governance architecture and compilation
- Formal Execution Semantics: theoretical foundation
- AGCP: execution-layer control plane

AGCP differentiates itself by defining explicit commit semantics, artifact-mediated execution, and lifecycle state derived from immutable history, bridging formal models and practical governance enforcement.

2.7 Identity, Authorization, and Trust Infrastructure

Several systems address identity, authorization, and trust in distributed environments, but do not define execution-layer governance semantics.

Zanzibar (Global Authorization Systems) Google Zanzibar (Pang et al., 2020) defines a globally consistent authorization system based on relationship tuples and access checks. It provides:

- fine-grained authorization decisions
- globally consistent access control
- low-latency permission evaluation

However, Zanzibar evaluates whether access is permitted but does not define how authorized actions are committed, how lifecycle state is managed, or how execution is gated. AGCP extends beyond authorization by introducing commit semantics and lifecycle-aware execution control.

SPIFFE/SPIRE (Workload Identity) SPIFFE and SPIRE (SPIFFE, 2022) provide cryptographically verifiable workload identity for distributed systems. They enable:

- secure identity issuance
- workload authentication
- identity-based policy enforcement

These systems address identity establishment and trust but do not define how state transitions are governed or committed. AGCP incorporates identity as an input to governance evaluation but defines additional layers of policy evaluation, authorization artifacts, and execution gating.

Sigstore (Software Supply Chain Integrity) Sigstore (Newman et al., 2022) provides mechanisms for signing, verifying, and recording software artifacts in transparency logs. It supports:

- artifact signing and verification
- transparency through append-only logs
- provenance tracking

While Sigstore establishes integrity and provenance of artifacts, it does not define governance semantics for execution of those artifacts. AGCP similarly uses append-only logs but applies them to governance decisions and lifecycle state derivation rather than artifact verification.

Positioning Relative to AGCP These systems address critical components of distributed trust:

- Zanzibar: authorization
- SPIFFE/SPIRE: identity
- Sigstore: integrity and provenance

AGCP operates at a different layer, governing the admissibility and execution of state transitions by integrating identity, authorization, and policy evaluation into a deterministic execution model with explicit commit semantics.

3 Problem Formulation and Control Plane Concept

3.1 Problem Statement

Given:

- a proposed state transition p
- canonical system state S
- governance constraints

Determine whether p may be committed as:

$$S \rightarrow S'$$

under the constraints of:

- deterministic evaluation
- invariant constraints
- authorization requirements
- lifecycle consistency

3.2 Control Plane Definition

AGCP is a control plane governing:

- admissibility
- authorization
- commit

It operates independently of:

- reasoning systems
- application logic
- execution systems

This separation is consistent with established control-plane/data-plane architectures in distributed systems (Kreutz et al., 2015; Burns et al., 2016).

3.3 Control Plane Position

The AGCP architecture can be expressed as:

$$\text{Reasoning} \rightarrow \text{AGCP Control Plane} \rightarrow \text{Execution}$$

- Reasoning proposes state transitions
- AGCP governs admissibility and authorization
- Execution applies only authorized transitions

The overall architecture of AGCP, including the separation between reasoning systems, the control plane, and execution systems, is illustrated in Figure 1.

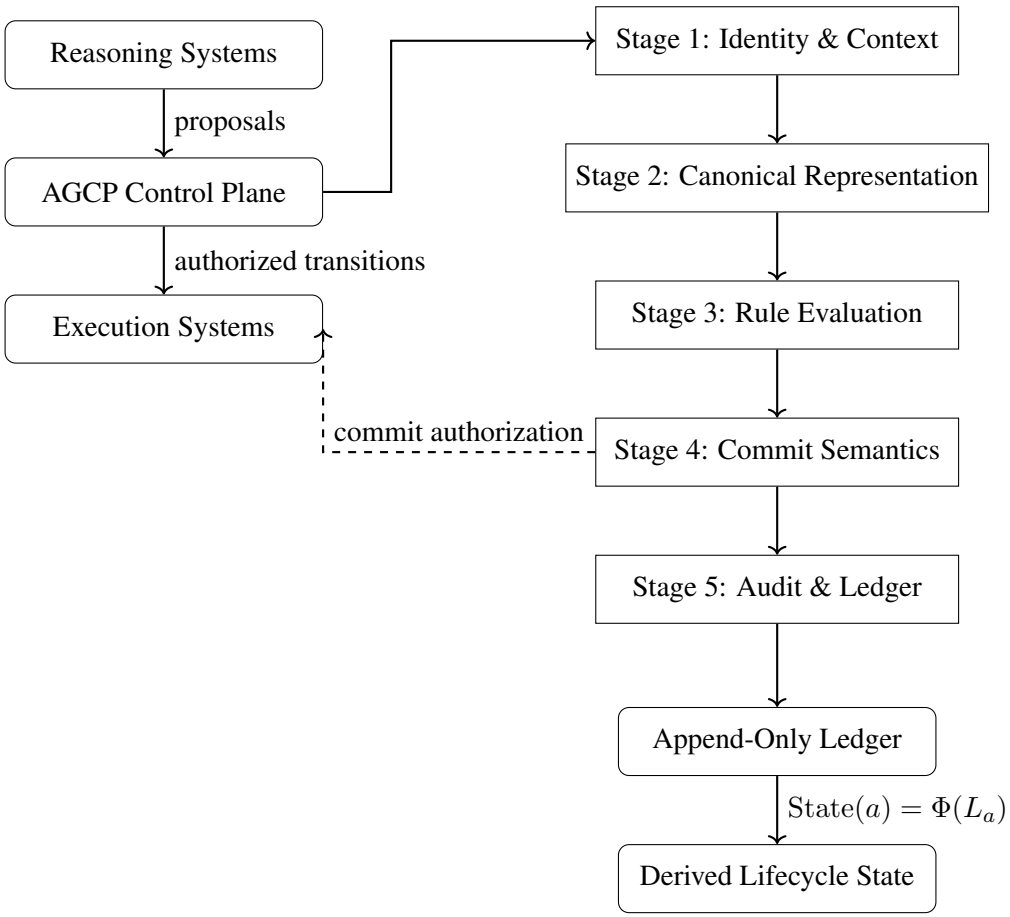

Figure 1: AGCP control plane architecture. Reasoning systems submit proposals to the AGCP control plane, which evaluates them through a five-stage deterministic pipeline. Execution systems receive only authorized transitions. Ledger entries produced by the pipeline serve as the basis for deterministic lifecycle-state derivation.

4 AGCP System Definition

AGCP is a deterministic governance control plane system that:

- receives proposed state transitions
- evaluates them against policy and invariants
- produces authorization artifacts
- gates execution based on lifecycle state
- records all decisions in an append-only ledger

AGCP differs from policy engines, identity systems, and workflow frameworks in that it defines execution governance at the point of state mutation rather than only evaluating permissions or orchestrating processes. These distinctions are examined in Section 11.

Although motivated by the behavior of autonomous agents, the AGCP model applies equally to human-initiated and programmatic system interactions, provided they result in proposed state transitions.

5 Separation of Reasoning and Execution

AGCP enforces a separation between proposal and execution:

> Reasoning proposes. Governance decides. Execution commits.

In the AGCP model:

- agents are not permitted to directly execute mutations
- all execution requires prior authorization
- authorization is bound to canonical state and context

This separation is intended to allow reasoning systems to remain flexible while ensuring that execution is governed by deterministic control-plane rules.

The increasing coupling of reasoning and action generation in modern systems (Yao et al., 2023), along with the need for external control mechanisms in AI systems (Russell, 2019), further motivates this architectural separation.

This design is also consistent with established security principles emphasizing separation of mechanism and policy (Saltzer and Schroeder, 1975).

6 Deterministic Governance Pipeline

The AGCP architecture defines a normative five-stage pipeline for governance evaluation and execution control. The structure of this pipeline and its relationship to execution and ledger recording are shown in Figure 1.

AGCP operates under the assumption that governance intent has been compiled into executable artifacts prior to entering the control plane. This compilation process, described in the Runtime Governance Architecture (RGA) (Willis, 2026c), produces the policy representations, constraints, and evaluation functions consumed by AGCP. As such, AGCP does not perform governance compilation, but instead enforces compiled governance deterministically at execution time.

AGCP can be interpreted as a deterministic state-machine architecture in which system state is derived from ordered execution history, consistent with established distributed systems models.

6.1 Stage 1 — Identity and Context Formation

This stage establishes the trust context under which the proposed action is evaluated. It:

- verifies identity
- validates trust in identity and credentials
- verifies provenance and integrity of submitted artifacts
- binds the request to tenant and contextual attributes

These checks ensure that subsequent governance evaluation operates on trusted inputs.

Typical Interfaces This stage typically interfaces with identity and trust infrastructure such as SPIFFE/SPIRE for workload identity, OIDC-based identity providers, certificate authorities, and provenance systems such as Sigstore for artifact integrity and verification.

6.2 Stage 2 — Canonical Representation

This stage:

- transforms input into a deterministic representation
- enforces schema constraints
- supports idempotency and replay (Kleppmann, 2017)

Typical Interfaces This stage typically interfaces with schema validation and normalization layers, including JSON Schema validation, canonicalization standards (e.g., RFC 8785), and API gateways or middleware responsible for enforcing deterministic input representations.

6.3 Stage 3 — Rule Evaluation (Policy Evaluation Contract, PEC)

This stage evaluates governance constraints using a Policy Evaluation Contract (PEC), which defines the deterministic evaluation interface for policy logic. It evaluates:

- policy constraints
- invariants
- human-in-the-loop (HITL) requirements

It produces a decision mapped to lifecycle state:

```
AUTHORIZED | REJECTED | PENDING_HITL
```

Typical Interfaces This stage typically interfaces with policy evaluation engines and decision frameworks such as Open Policy Agent (OPA), attribute-based access control (ABAC) systems, Zanzibar-like authorization systems, and custom policy evaluation contracts defined within governance frameworks.

6.4 Stage 4 — Commit Semantics (Execution Boundary)

This stage defines the execution boundary.

Execution is permitted only if:

- lifecycle state = AUTHORIZED
- a valid authorization reference exists
- tenant state is valid

This approach is consistent with transaction commit semantics in distributed systems (Gray and Reuter, 1993) and reduces risks associated with separating authorization from execution (Bishop and Dilger, 1996).

Typical Interfaces This stage typically interfaces with execution control mechanisms, including transactional systems, orchestration layers, admission controllers (e.g., Kubernetes admission control), and enforcement points that gate execution based on authorization state.

6.5 Stage 5 — Audit and Lifecycle Recording

This stage:

- records append-only ledger entries
- preserves ordering
- enables deterministic replay

The use of ordered records to support deterministic reconstruction is consistent with distributed system models of event ordering and replay (Lamport, 1978).

Typical Interfaces This stage typically interfaces with append-only logging and audit infrastructure, including event streaming systems (e.g., Kafka), distributed logs, transparency logs, and ledger or event-sourcing systems used for immutable record-keeping.

These interfaces illustrate that AGCP operates as a control-plane layer integrating existing identity, policy, execution, and logging systems into a unified governance model.

7 AGCP Artifact Model (Normative)

AGCP is an artifact-centric system, meaning all interactions are mediated through structured, immutable artifacts. The normative definitions of these artifacts, including their schemas and constraints, are specified in the AGCP Specification v0.9.0 (Willis, 2026a), which includes a publicly available reference repository.

All artifacts in AGCP conform to the following normative properties:

- Artifacts MUST be immutable once created
- Artifacts MUST be uniquely identifiable within a tenant scope
- Artifacts MUST be bound to the context under which they were evaluated
- Artifacts MUST be suitable for deterministic replay and verification

These properties ensure that artifact interactions preserve determinism, auditability, and consistency across the governance pipeline.

Artifacts constitute the sole mechanism through which state transitions are proposed, evaluated, authorized, and recorded within AGCP.

This design enables traceability, verifiability, and deterministic reconstruction of system behavior, consistent with established work in data provenance (Buneman et al., 2001; Cheney et al., 2009).

7.1 Governance Action Envelope

The Governance Action Envelope is the canonical input artifact. It contains:

- identity information
- mutation attributes
- policy context
- provenance data

This artifact represents a proposed state transition in a deterministic and schema-constrained form. It is a structured representation defined by a normative schema and may be serialized using formats such as JSON or transmitted via protocols such as the Model Context Protocol (MCP), provided that deterministic representation and validation constraints are preserved.

7.2 Authorization Reference

The Authorization Reference is a derived artifact representing proof of admissibility. It is a verifiable, context-bound reference to a completed governance evaluation, suitable for consumption by execution systems.

The reference may take the form of a structured token, identifier, or cryptographically bound artifact reference, provided it uniquely identifies the evaluated action and binds it to the context under which authorization was granted.

It is:

- scoped to a specific action
- non-reusable
- bound to evaluated context
- verifiable by execution systems

Execution systems rely on this reference to determine whether a transition is eligible for commit.

7.3 Ledger Entry

Each stage in the pipeline produces a ledger entry of the form:

$$(\text{tenant\_id}, \text{action\_id}, \text{seq}, \text{stage}, \text{data}, [\text{optional hash}])$$

where `seq` denotes a monotonically increasing sequence number defining the order of entries associated with a given action.

Properties include:

- append-only structure
- total ordering per action
- immutability

Total ordering per action means that all ledger entries associated with a given action identifier are ordered deterministically according to their sequence number, forming a linear history L_a used for lifecycle derivation.

The optional hash field may be used to provide cryptographic integrity guarantees. In particular, it may encode a hash of the current entry or a chained hash of the previous entry, forming a hash-linked sequence of ledger entries. This enables detection of tampering or reordering within the ledger history.

While hash chaining is not required for the abstract model, it provides an implementation mechanism for strengthening auditability and integrity guarantees.

The use of ordered records to support deterministic reconstruction is consistent with distributed system models of event ordering and replay (Lamport, 1978).

7.4 HITL Tokens

Human-in-the-loop (HITL) tokens represent externally provided approvals required for certain governance decisions.

They are:

- cryptographically bound
- tenant-scoped
- optionally quorum-based

HITL tokens are bound to a specific action and its evaluated context, and therefore cannot be reused across different actions or contexts.

A HITL token attests that an external approval has been granted for a proposed state transition. Tokens MUST be verifiable by the control plane and MUST correspond to the action and context under evaluation.

In cases where quorum is required, multiple HITL tokens are evaluated collectively, and authorization is granted only when the quorum condition is satisfied.

HITL tokens are consumed during Stage 3 (Rule Evaluation) and contribute to the transition of an action into the `AUTHORIZED` lifecycle state.

8 Commit Semantics and Execution Gating

AGCP defines an explicit commit boundary at the execution layer.

8.1 Commit Rule

The AUTHORIZED lifecycle state is assigned when the governance evaluation in Stage 3 produces an authorization decision and all required conditions, including policy constraints, invariants, and any required HITL approvals, have been satisfied.

A state transition is valid only if:

$$\text{State}(a) = \text{AUTHORIZED} \quad \text{and} \quad \text{authorization\_reference exists}$$

This rule defines execution eligibility as a function of admissibility, where admissibility is determined during governance evaluation and represented by the AUTHORIZED lifecycle state.

Authorization in AGCP is enforced at execution time rather than as a prior check, ensuring that admissibility and execution remain tightly coupled.

This rule binds authorization directly to execution eligibility.

8.2 Execution Definition

Execution is defined as a committed state transition. That is, execution occurs only when a transition crosses the commit boundary. Execution results in a terminal lifecycle state denoted as EXECUTED.

As shown in Figure 2, execution is reachable only from the AUTHORIZED state, making commit eligibility an explicit lifecycle property rather than an implicit consequence of policy evaluation.

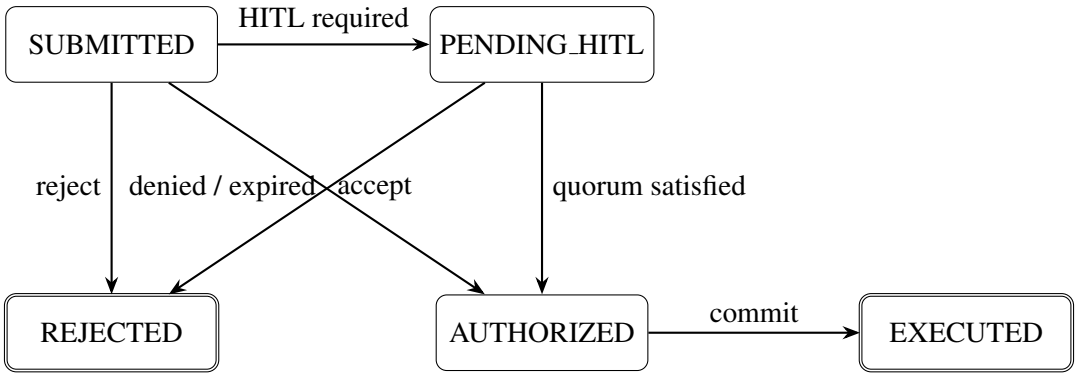

Figure 2: AGCP action lifecycle state machine. The model distinguishes proposal submission, human-approval waiting, authorization, rejection, and execution. REJECTED and EXECUTED are terminal states. Execution is reachable only from AUTHORIZED.

8.3 Execution Gating

Transitions that do not satisfy the commit rule are not eligible for execution within the system model.

This approach aligns with transaction commit semantics in distributed systems (Gray and Reuter, 1993) and reduces risks associated with separating validation and execution, including TOCTOU inconsistencies (Bishop and Dilger, 1996).

8.4 Execution Mediation

Execution systems are required, under the AGCP model, to validate authorization artifacts prior to applying mutations.

This requirement is consistent with enforceable policy models in which execution must be mediated by a trusted mechanism (Schneider, 2000).

9 Ledger-Derived State Model

AGCP defines lifecycle state as a function exclusively derived from ledger history rather than maintained as mutable system state.

9.1 Lifecycle Function

Let L_a denote the ordered ledger entries associated with action a.

Then:

$$\text{State}(a) = \Phi(L_a)$$

where Φ is a deterministic lifecycle derivation function.

The AGCP action lifecycle is illustrated in Figure 2. The figure shows the permitted transitions between submission, human-approval, authorization, rejection, and execution states.

9.2 Implications

This model has several implications:

- reduces the risk of state drift relative to mutable state models (Kleppmann, 2017)
- enables deterministic reconstruction of lifecycle state
- removes dependence on mutable status fields

9.3 Ledger as System of Record

Under this model, the ledger functions as the canonical source of lifecycle state.

Rather than storing state directly, the system derives it from ordered historical records, consistent with distributed system models of event ordering and reconstruction (Lamport, 1978).

Figure 1 illustrates how lifecycle state is derived from the append-only ledger rather than maintained as mutable system state.

Figure 2 presents the abstract lifecycle model, while the ledger-derived realization of that lifecycle is defined by the mapping $\text{State}(a) = \Phi(L_a)$.

10 Interface and Protocol Model

AGCP exposes a structured interface for interacting with the governance control plane.

10.1 Core Operations

The interface includes the following operations:

- `submit_action(envelope)`
- `evaluate_action(action_id)`
- `authorize_action(action_id)`
- `commit_action(action_id)`
- `query_ledger(action_id)`

These operations correspond directly to stages in the governance pipeline and operate on well-defined artifacts.

10.2 Interface Properties

The interface satisfies the following properties:

- deterministic input-output behavior
- idempotent submission semantics
- cryptographic binding between artifacts and evaluation context

Idempotency and replayability are consistent with distributed system design practices for reliable processing (Kleppmann, 2017).

10.3 Execution Integration

Execution systems are required, under the AGCP model, to validate authorization artifacts prior to applying mutations.

This requirement ensures that execution remains mediated by the control plane and is consistent with enforceable policy models (Schneider, 2000).

11 Conformance and Determinism

AGCP defines conformance levels (L1–L5) as specified in the AGCP conformance specification (Willis, 2026a). These levels represent increasing degrees of determinism, completeness, and enforcement:

- **L1** — Basic structural conformance (artifact and schema validity)
- **L2** — Deterministic processing of governance inputs
- **L3** — Deterministic policy evaluation and lifecycle state assignment
- **L4** — Enforcement of commit semantics and execution gating
- **L5** — Full determinism including replayability, auditability, and lifecycle reconstruction

11.1 Determinism Requirement

For a given proposal p under identical conditions:

$$G(p) = G(p)$$

That is, identical inputs must produce identical outputs.

This property is consistent with deterministic state-machine models used in distributed systems (Lamport, 1978).

11.2 Replay Requirement

Replay of a proposal under identical conditions produces identical ledger output:

$$\text{Replay}(p) \rightarrow L_p$$

where L_p is the resulting ledger sequence.

This requirement aligns with deterministic replication models such as Raft (Ongaro and Ousterhout, 2014).

11.3 Implications

These properties ensure:

- reproducibility of governance decisions
- auditability through replay
- consistency across distributed deployments

12 Comparative Systems Analysis

While AGCP is motivated by the need to govern autonomous agent behavior, it is not limited to AI-based systems. The model applies to any system in which state transitions must be evaluated and controlled, including traditional software systems, infrastructure automation, and distributed services.

AGCP integrates capabilities traditionally provided by identity systems, policy engines, Zero Trust architectures, and workflow systems into a unified execution-layer control plane.

12.1 Policy Engines (e.g., Open Policy Agent (OPA))

Policy engines such as Open Policy Agent (OPA) perform attribute-based policy evaluation (Hu et al., 2015).

They:

- evaluate conditions based on input attributes
- return allow/deny decisions
- do not natively define lifecycle state or commit semantics

Within AGCP, policy engines such as OPA may be used as components of the Policy Evaluation Contract (PEC) in Stage 3, providing the evaluation logic over governance inputs. In this role, they operate as deterministic decision functions within the broader governance pipeline.

AGCP extends these systems by binding evaluation outcomes to lifecycle state, authorization artifacts, and execution eligibility through explicit commit semantics.

12.2 IAM / RBAC / ABAC Systems

Identity and access management systems define relationships between users, roles, and permissions (Sandhu et al., 1996).

These systems:

- define who is authorized to perform actions
- provide identity and attribute context

Within AGCP, IAM systems provide identity and attribute inputs to Stage 1 (Identity and Context Formation) and Stage 3 (Rule Evaluation). AGCP extends these systems by governing how authorization decisions are bound to execution through lifecycle state and commit semantics.

12.3 Zero Trust Architectures

Zero Trust architectures emphasize continuous verification of identity and context (Rose et al., 2020).

They:

- enforce identity and context validation
- support dynamic access control decisions

Within AGCP, Zero Trust principles are realized primarily in Stage 1, where identity, trust, and contextual attributes are validated prior to governance evaluation. AGCP extends these principles by binding verified context to deterministic policy evaluation and execution gating.

12.4 Workflow Systems

Workflow systems coordinate processes and state progression using predefined control-flow patterns (van der Aalst et al., 2003).

They:

- manage task sequencing and orchestration
- rely on mutable state representations

Within AGCP, workflow-like coordination may be represented through sequences of governance actions, but lifecycle state is derived from immutable ledger history rather than maintained as mutable workflow state. AGCP extends workflow systems by enforcing execution gating based on authorization state and deterministic replay of transitions.

13 Architectural Positioning within the Governance Stack

AGCP operates as the execution-layer control plane within a broader governance stack. It is not a standalone governance system, but a component responsible for enforcing governance decisions at the point of execution.

Within this stack:

- The Practitioner's Blueprint for Secure Artificial Intelligence (PBSAI) (Willis, 2026d) defines the governance ecosystem, including domains, actors, and policy sources.
- The Runtime Governance Architecture (RGA) (Willis, 2026c) defines the compilation of governance intent into executable artifacts.
- Formal execution semantics (Willis, 2026b) define the theoretical properties of deterministic evaluation, invariants, and safety guarantees.
- AGCP (Willis, 2026a) implements the execution-layer enforcement of these compiled governance artifacts through a deterministic control plane.

This separation of concerns ensures that governance intent, compilation, and execution are handled by distinct components with clearly defined responsibilities.

AGCP assumes that governance artifacts have already been compiled by RGA and focuses exclusively on evaluating and enforcing those artifacts at runtime. In this role, AGCP binds admissibility decisions to execution through explicit commit semantics and lifecycle state transitions.

This layered model enables composability, allowing governance policies defined at higher levels to be enforced consistently across heterogeneous systems through a unified execution model.

14 Formal Model (Condensed) and Threat Model

This formulation is consistent with state-machine replication models in which system behavior is defined as a deterministic function over an ordered log of inputs, as formalized in (Willis, 2026b). AGCP can be understood as a concrete system-level realization of these formal execution semantics.

In addition to the formal model, AGCP defines a set of system-level properties that characterize its behavior under deterministic execution (Willis, 2026b).

14.1 Condensed Formal Model

AGCP implements a deterministic pipeline:

$$G = f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$$

where each function corresponds to a stage in the governance pipeline.
State transition behavior is defined as:

$$\Omega(p, S, L) \to S' \text{ or } \bot$$

where:

- p is a proposed state transition
- S is the current system state
- L is the ledger history

Lifecycle state is derived as:

$$\text{State}(a) = \Phi(L_a)$$

Formal guarantees, including determinism, invariant preservation, and non-bypassability, are defined in prior work (Willis, 2026b).

14.2 Properties and Guarantees

The AGCP model defines a set of system-level properties that follow from its deterministic pipeline, artifact model, and ledger-derived state representation. These properties are stated at the level of system behavior rather than empirical performance.

14.2.1 Determinism

Let G denote the deterministic governance pipeline. For any proposal p, canonical system state S, and ledger history L, consider two independent evaluations of G under identical conditions:

$$G_1(p, S, L), \quad G_2(p, S, L)$$

Then:

$$G_1(p, S, L) = G_2(p, S, L)$$

That is, identical inputs produce identical outputs.

Identical inputs include the proposal p, canonical state S, ledger history L, and all deterministic evaluation context, including policy modules, constraint definitions, invariant specifications, and tenant-scoped configuration.

This property ensures that governance decisions are reproducible and independent of execution environment variability. Determinism is a necessary condition for reliable audit, replay, and verification within the AGCP model.

14.2.2 Replay Equivalence

Let L_p denote the ledger sequence produced by evaluating proposal p. Then:

$$\text{Replay}(p) \rightarrow L_p$$

That is, replaying a proposal under identical conditions produces an identical sequence of ledger entries. This property enables deterministic reconstruction of system behavior and is consistent with log-based state-machine models.

14.2.3 Non-Bypassability (Under Assumption)

Assuming all execution paths are mediated through the AGCP control plane, no state transition can be committed without satisfying the commit rule:

$$\text{State}(a) = \text{AUTHORIZED}$$

and the existence of a valid authorization reference.

This property ensures that governance constraints cannot be bypassed through direct execution paths, provided that execution systems enforce control-plane mediation.

AGCP does not itself enforce that all execution paths are mediated through the control plane. Instead, it defines the conditions under which non-bypassability holds, assuming that external execution systems correctly enforce mediation.

Violation of this assumption may permit unauthorized state transitions outside the governance model.

14.2.4 Authorization Soundness

A committed state transition implies that:

- the transition has passed rule evaluation via the Policy Evaluation Contract (PEC)
- all evaluated invariants have been satisfied under the governing policy
- any required human approvals have been obtained

Within AGCP, the Policy Evaluation Contract (PEC) defines a deterministic evaluation interface that produces an admissibility decision for a proposed state transition. This evaluation may incorporate attribute-based inputs (e.g., ABAC) and may be implemented using policy decision point (PDP) mechanisms; however, these are abstracted within the PEC and do not define its semantics.

The PEC subsumes the role of a PDP by integrating policy evaluation, invariant checking, and human approval conditions into a single admissibility decision represented as lifecycle state.

This property follows from the binding between lifecycle state and execution eligibility.

14.2.5 State Derivation Consistency

Lifecycle state is defined as:

$$\text{State}(a) = \Phi(L_a)$$

Because L_a is append-only and ordered, and Φ is deterministic, lifecycle state is uniquely determined by ledger history. This eliminates ambiguity arising from mutable state representations.

14.2.6 Idempotency of Submission

Given canonical representation at Stage 2, repeated submission of identical proposals results in equivalent evaluation outcomes and ledger sequences, provided the surrounding context and state remain unchanged.

This property supports safe retry semantics in distributed environments.

14.2.7 TOCTOU Mitigation (Structural)

AGCP structurally mitigates time-of-check/time-of-use (TOCTOU) inconsistencies by binding authorization to commit semantics and requiring validation at execution time. Authorization is not treated as a prior check but as a prerequisite condition that must hold at the commit boundary.

This eliminates classes of TOCTOU vulnerabilities arising from decoupling between authorization and execution, as execution is permitted only when authorization remains valid relative to the current canonical state.

However, complete elimination of TOCTOU inconsistencies depends on additional assumptions, including atomic execution within the target system and full mediation of all execution paths through the control plane. In the absence of these conditions, AGCP reduces but does not universally eliminate TOCTOU risk.

14.2.8 Auditability

Because all decisions and transitions are recorded in an append-only ledger:

- all actions are traceable
- lifecycle state can be reconstructed
- governance decisions can be independently verified

Auditability follows directly from the combination of determinism and immutable record-keeping.

14.2.9 Compositionality

The governance pipeline is defined as a composition of deterministic functions:

$$G = f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1$$

This allows reasoning about system behavior at the level of individual stages while preserving end-to-end determinism.

14.2.10 Scope of Guarantees

These properties hold under the assumptions defined in the threat model, including:

- correct implementation of deterministic evaluation
- integrity of the append-only ledger
- enforcement of execution mediation

Violation of these assumptions may invalidate one or more guarantees.

14.3 Concurrency Semantics

AGCP operates in environments where multiple proposals may be submitted concurrently and may target overlapping portions of the execution state. The model does not assume serialized proposal submission and explicitly permits concurrent evaluation of independent proposals.

14.3.1 Concurrent Proposal Evaluation

Let $p_1, p_2, \ldots, p_n$ denote a set of proposals submitted concurrently.

Each proposal is evaluated independently through the deterministic governance pipeline:

$$G(p_i, S, L) \rightarrow d_i$$

where:

- S is the canonical system state at evaluation time
- L is the ledger history
- $d_i \in \{\text{AUTHORIZED}, \text{REJECTED}, \text{PENDING\_HITL}\}$

Evaluation is performed relative to a snapshot of canonical state. As a result, multiple concurrent proposals may be evaluated as AUTHORIZED even if they target the same underlying resource or would produce conflicting state transitions.

The AGCP model does not require conflict detection or serialization at evaluation time.

14.3.2 Separation of Evaluation and Commitment

AGCP separates:

- logical admissibility, determined during governance evaluation
- physical state mutation, which occurs only at the commit boundary

Admissibility is therefore necessary but not sufficient for execution.

A proposal p is eligible for execution only if:

$$\begin{aligned} \text{State}(a) = \text{AUTHORIZED} \\ \wedge\ \exists\, \text{AuthRef}(a) \\ \wedge\ \text{admissible}(p, S_{\text{current}}) \end{aligned}$$

This ensures that execution eligibility is determined at commit time rather than at evaluation time.

14.3.3 Commit-Ordered Execution

Execution of state transitions is governed by commit ordering.

Let p_i and p_j be two proposals evaluated concurrently and both determined to be AUTHORIZED. Let their corresponding actions be a_i and a_j.

Execution proceeds as a sequence of commit attempts:

$$\text{commit}(a_i) \rightarrow \{\text{success}, \text{failure}\}$$

$$\text{commit}(a_j) \rightarrow \{\text{success}, \text{failure}\}$$

The outcome is determined by the order in which proposals reach the commit boundary and successfully satisfy execution conditions.

Commit ordering is defined by the sequence of successful execution commits recorded in the ledger.

14.3.4 Revalidation at Commit

At the commit boundary, execution eligibility is evaluated against the current canonical state derived from ledger history:

$$S_{\text{current}} = \Phi(L)$$

A proposal that was previously admissible relative to an earlier state snapshot may fail at commit time if the canonical state has changed due to prior commits.

Formally, a commit succeeds only if:

$$\text{commit}(a_i) \text{ succeeds only if admissible}(p_i, S_{\text{current}})$$

This revalidation ensures that execution reflects the most recent state rather than the state observed at evaluation time.

14.3.5 Optional Pre-Commit Binding Validation

Some AGCP implementations may introduce an additional validation step immediately prior to commit, referred to as *binding validation*. This step performs pre-commit arbitration among competing proposals that have been independently evaluated and determined to be admissible.

Binding validation addresses the question:

Given a set of admissible proposals, which proposal is permitted to become real?

Let $A = \{p_1, p_2, \ldots, p_n\}$ denote a set of proposals that have been evaluated as AUTHORIZED relative to a prior canonical state snapshot. Although each $p_i \in A$ is admissible, the proposals may not be mutually compatible when considered relative to the current canonical state.

Binding validation defines a selection function over admissible proposals:

$$\text{Select}(A, S_{\text{current}}) \subseteq A$$

where $S_{\text{current}} = \Phi(L)$ is the current canonical state derived from the ledger. The function Select determines which proposals remain valid for commitment under current conditions.

Equivalently, binding validation may be expressed as a predicate:

$$\text{Bindable}(p, S_{\text{current}}, A) \in \{\text{true}, \text{false}\}$$

Execution commit is permitted only if:

$$\text{State}(a) = \text{AUTHORIZED} \ \wedge \ \exists\, \text{AuthRef}(a) \ \wedge \ \text{Bindable}(p, S_{\text{current}}, A)$$

Binding validation operates after governance evaluation (including PEC execution) but before execution commit. The PEC determines admissibility relative to a snapshot of canonical state, whereas binding validation determines whether an admissible proposal can be reconciled with the current canonical state and competing proposals at commit time.

This step enables implementations to enforce:

- conflict resolution across competing admissible proposals
- resource-level exclusivity constraints
- reservation or locking semantics

The inclusion of binding validation is optional and does not alter the core AGCP model, which requires only that execution eligibility be validated at the commit boundary. Implementations that omit this step still preserve correctness through commit-time validation against canonical state.

Binding validation therefore provides a structured mechanism for selecting which admissible proposals may become committed state transitions, while preserving the separation between admissibility and binding.

14.3.6 Conflict Resolution Model

AGCP resolves conflicts between concurrent proposals through commit-time validation rather than evaluation-time coordination.

This behavior emerges from commit-time validation rather than explicit coordination during evaluation.

As a result:

- Multiple proposals may be simultaneously AUTHORIZED
- Only those that remain valid at commit time are executed
- Conflicting transitions are prevented by commit-time rejection

This model can be characterized as:

optimistic concurrency with deterministic commit validation

14.3.7 Ordering Guarantees

For a given action a, ledger entries form a totally ordered sequence:

$$L_a = [(\text{seq}_1, \text{stage}_1), (\text{seq}_2, \text{stage}_2), \dots]$$

where:

$$\text{seq}_{k+1} > \text{seq}_k$$

Ordering is scoped per action identifier. No global total ordering across actions is required.

Commit ordering across actions is determined by the sequence of successful execution commits as observed in the system.

14.3.8 Safety Implications

This concurrency model ensures:

- Deterministic outcomes: execution results depend only on commit order and canonical state
- Conflict safety: incompatible transitions cannot both be committed
- Replay consistency: identical inputs and commit ordering produce identical ledger histories
- TOCTOU mitigation: execution is validated at the point of commit

14.3.9 Summary

AGCP does not enforce first-come-first-served semantics. Instead, it enforces:

commit-ordered, state-validated execution

Concurrent proposals are evaluated independently but serialized at the commit boundary, where execution eligibility is determined relative to the current canonical state.

14.4 Threat and Failure Model

14.4.1 Adversary Capabilities

An adversary may attempt to:

- submit malformed or adversarial proposals
- replay previously valid actions
- manipulate context or identity attributes
- substitute execution targets

These attack classes are consistent with those defined in formal analyses of governance control planes (Willis, 2026b).

14.4.2 Failure Modes

Potential system failures include:

- nondeterministic policy evaluation
- incorrect ledger ordering
- improper invariant enforcement
- execution based on authorization artifacts that are not revalidated at commit time

14.4.3 Failure Handling and Response

AGCP defines deterministic failure handling behavior across the governance pipeline. Failures are handled in a structured manner consistent with the pipeline stages and the ledger-derived lifecycle model.

Failure Classification Failures are classified based on where they occur in the governance pipeline:

- **Structural failures**: Failures occurring before or during early pipeline stages (e.g., schema validation failure, provenance verification failure, invalid tenant state)
- **Evaluated failures**: Failures resulting from governance evaluation (e.g., policy rejection, invariant violation, unmet HITL requirements)
- **Commit-time failures**: Failures occurring at the commit boundary (e.g., stale authorization, invalid state at commit, binding validation failure)

Structural Failure Handling Structural failures prevent a proposal from being admitted into the governance evaluation process.

In such cases:

- the proposal is rejected immediately
- no lifecycle state is assigned
- no governance decision is produced

These failures indicate that the proposal is not valid for evaluation within the control plane.

Evaluated Failure Handling Evaluated failures occur during deterministic governance evaluation (e.g., within the Policy Evaluation Contract).

In these cases:

- a deterministic decision is produced (e.g., REJECTED or PENDING_HITL)
- the outcome is recorded in the ledger
- lifecycle state is derived accordingly

This ensures that all evaluated proposals are auditable and replayable.

Commit-Time Failure Handling Commit-time failures occur when a previously admissible proposal is no longer valid relative to the current canonical state.

In such cases:

- execution is denied
- no state mutation is applied to the execution target
- the proposal does not transition to EXECUTED

These failures arise from revalidation at the commit boundary and include conditions such as stale authorization artifacts or state conflicts.

Determinism Requirement Failure handling must be deterministic. For identical inputs and system conditions, failure outcomes must be reproducible:

$$G(p, S, L) = G(p, S, L)$$

This includes consistent classification of failures, lifecycle outcomes, and ledger records where applicable.

Safety Implications This failure handling model ensures:

- no partial or undefined state transitions
- consistent auditability of both successful and failed proposals
- preservation of invariants under failure conditions

Failures therefore result in controlled, observable outcomes rather than undefined behavior.

14.4.4 Mitigations

AGCP mitigates these risks through:

- deterministic evaluation requirements
- append-only, ordered ledger constraints
- authorization validation at commit time
- canonical representation for idempotency

14.4.5 Out-of-Scope Threats

The model does not address:

- compromise of cryptographic primitives
- compromise of control-plane infrastructure
- side-channel or physical-layer attacks

14.4.6 Critical Assumption

All execution paths must be mediated through the control plane.

If execution bypasses the control plane, the guarantees defined by the model do not hold (Willis, 2026b).

14.5 Worked Example: HITL-Governed Transition

Consider a proposed state transition requiring human approval.

1. A proposal p is submitted via a Governance Action Envelope
2. Stage 3 evaluation returns `PENDING_HITL`
3. A ledger entry is recorded for this decision
4. Human reviewers provide signed HITL tokens
5. A quorum condition is satisfied
6. An authorization reference is generated
7. A ledger entry records `EXECUTION_AUTHORIZED`
8. Lifecycle state becomes AUTHORIZED via ledger derivation
9. The execution system validates the authorization reference
10. The transition is committed, producing `EXECUTION_COMMITTED`

The lifecycle state of the action is derived from the ledger as:

$$\text{State}(a) = \text{EXECUTED}$$

This example illustrates:

- pipeline progression across stages
- artifact interaction
- lifecycle derivation from ledger history
- execution gating based on authorization

15 Performance Considerations

AGCP introduces additional processing stages in the execution path of state transitions. These stages may introduce latency relative to systems that permit direct execution without governance mediation. This section characterizes the performance impact attributable specifically to AGCP, independent of underlying systems such as identity providers, policy engines, or execution infrastructure, which are assumed to be present in comparable architectures.

15.1 Scope of Overhead

The performance impact considered here is limited to overhead introduced by the AGCP control plane itself. This includes:

- deterministic pipeline orchestration
- artifact construction and validation
- lifecycle state derivation from ledger history
- commit-time validation and execution gating

Interactions with external systems (e.g., identity verification, policy evaluation engines, execution targets) are treated as a baseline cost and are not attributed to AGCP-specific overhead.

15.2 Sources of Latency

AGCP introduces latency primarily through the following mechanisms:

Pipeline Serialization The governance pipeline enforces a fixed ordering of stages. While individual stages may be efficient, the requirement that they execute in sequence introduces inherent serialization overhead.

Artifact Processing The construction, validation, and verification of governance artifacts (e.g., envelopes, authorization references, ledger entries) introduce additional processing steps compared to systems that operate on implicit or mutable state.

Ledger Interaction Append-only ledger recording and lifecycle state derivation introduce overhead associated with:

- write operations for stage recording
- read operations for state derivation

The extent of this overhead depends on the underlying storage implementation and indexing strategy.

Commit-Time Validation Revalidation at the commit boundary, including authorization checks and optional binding validation, introduces additional latency immediately prior to execution. This step is necessary to ensure correctness under concurrent state transitions.

15.3 Latency Characteristics

The latency introduced by AGCP exhibits the following characteristics:

- **Deterministic structure**: The number and order of processing steps are fixed, enabling predictable latency profiles.
- **Bounded overhead**: Each stage performs a well-defined and finite set of operations.

- **Composition over duplication**: AGCP composes existing system checks rather than duplicating their functionality.

As a result, latency can be modeled as the sum of stage-specific costs rather than an unbounded or emergent property.

15.4 Trade-Offs

AGCP introduces a trade-off between execution latency and governance guarantees:

- increased latency due to structured validation and recording
- improved determinism, auditability, and safety

In particular, commit-time validation ensures correctness under concurrency but introduces additional delay relative to systems that perform only pre-execution checks.

15.5 Optimization Considerations

Implementations may reduce latency through:

- efficient artifact serialization and validation
- optimized ledger storage and indexing
- caching of derived state where consistent with determinism
- parallelization of independent preprocessing steps

Such optimizations must preserve deterministic behavior and replayability.

15.6 Summary

AGCP introduces bounded and structurally predictable latency associated with deterministic governance enforcement. This overhead is a direct consequence of enforcing commit-time validation, artifact-mediated interaction, and ledger-derived state, and reflects an intentional trade-off between performance and verifiable execution correctness.

16 Limitations

The AGCP model has several limitations:

- requires a deterministic evaluation environment
- introduces latency due to multi-stage evaluation
- depends on correct specification of invariants
- does not guarantee correctness of policy logic
- assumes that execution cannot bypass the control plane

17 Conclusion

This paper introduced AGCP as a deterministic execution-layer governance control plane.

AGCP defines:

- a five-stage governance pipeline
- an artifact-centric interaction model
- explicit commit semantics governing execution
- a ledger-derived lifecycle model
- execution gating based on authorization state

The architecture enforces a separation between reasoning, governance, and execution, and was positioned relative to existing systems and within a broader architectural stack including PBSAI, RGA, and formal execution semantics.

AGCP operates on the assumption that governance intent has been compiled into executable artifacts prior to entering the control plane. It does not perform governance compilation, but instead enforces compiled policies, constraints, and invariants deterministically at execution time.

Together, these components define a framework for deterministic and verifiable governance of system state transitions, in which admissibility is evaluated, binding is enforced at the commit boundary, and execution is permitted only under validated conditions.

Copyright

Code and Specification Availability

The Artificial Intelligence Governance Control Plane (AGCP) specification and related artifacts referenced in this paper are publicly available.

AGCP Specification (archived release):
https://doi.org/10.5281/zenodo.18923302

AGCP Reference Repository:
https://github.com/jwillisSFT/agcp-spec

Patent Notice

Elements of the architecture described in this work may be covered by pending patent applications. Publication of this paper does not grant a license to any patent rights associated with the described systems or methods. Development and pre–sales use of the core specification is freely encouraged; however, commercialization is subject to licensing.

References

Bishop, M., and Dilger, M. (1996). Checking for Race Conditions in File Accesses. *Computing Systems*, 9(2), 131–152. Available at: https://www.usenix.org/legacy/publications/compsystems/1996/spr_bishop.pdf

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50–57. DOI: https://doi.org/10.1145/2890784

Buneman, P., Khanna, S., and Tan, W.-C. (2001). Why and Where: A Characterization of Data Provenance. In *Proceedings of the 8th International Conference on Database Theory (ICDT)*, 316–330. DOI: https://doi.org/10.1007/3-540-44503-X_20

Cheney, J., Chiticariu, L., and Tan, W.-C. (2009). Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases*, 1(4), 379–474. DOI: https://doi.org/10.1561/1900000006

Gray, J., and Reuter, A. (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann. Available at: https://books.google.com/books/about/Transaction_Processing.html?id=VFKbCgAAQBAJ

Hu, V. C., Ferraiolo, D. F., Kuhn, D. R., Schnitzer, A., Sandlin, K., Miller, R., and Scarfone, K. (2015). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. NIST Special Publication 800-162. Available at: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-162.pdf

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. Available at: https://dataintensive.net/

Kreutz, D., Ramos, F. M. V., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., and Uhlig, S. (2015). Software-Defined Networking: A Comprehensive Survey. *Proceedings of the IEEE*, 103(1), 14–76. DOI: https://doi.org/10.1109/JPROC.2014.2371999

Lamport, L. (1978). Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7), 558–565. DOI: https://doi.org/10.1145/359545.359563

Ongaro, D., and Ousterhout, J. (2014). In Search of an Understandable Consensus Algorithm (Raft). In *USENIX Annual Technical Conference*, 305–319. Available at: https://raft.github.io/raft.pdf

Pang, R., et al. (2020). *Zanzibar: Google's Consistent, Global Authorization System*. USENIX ATC. Available at: https://research.google/pubs/pub48190/

Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). *Zero Trust Architecture*. NIST Special Publication 800-207. Available at: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf

Russell, S. (2019). *Human Compatible: Artificial Intelligence and the Problem of Control. Viking.*

Saltzer, J. H., and Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), 1278–1308. DOI: https://doi.org/10.1109/PROC.1975.9939

Sandhu, R., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996). Role-Based Access Control Models. *IEEE Computer*, 29(2), 38–47. DOI: https://doi.org/10.1109/2.485845

Schneider, F. B. (2000). Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1), 30–50. DOI: https://doi.org/10.1145/353323.353382

Newman, Z., Meyers, J. S., and Torres-Arias, S. (2022). *Sigstore: Software Signing for Everybody*. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*, 2353–2367. DOI: https://doi.org/10.1145/3548606.3560596

SPIFFE Project (2022). *The SPIFFE and SPIRE Project*. Available at: https://spiffe.io

van der Aalst, W. M. P., ter Hofstede, A. H. M., Kiepuszewski, B., and Barros, A. P. (2003). Workflow Patterns. *Distributed and Parallel Databases*, 14(1), 5–51. DOI: https://doi.org/10.1023/A:1022883727209

Willis, J. M. (2026a). *Artificial Intelligence Governance Control Plane (AGCP) Specification v0.9.0*. Sustainable Future Tech Publishing. DOI: https://doi.org/10.5281/zenodo.18923302. Available at: https://github.com/jwillisSFT/agcp-spec

Willis, J. (2026b). *Formal Execution Semantics and Safety Invariants for Governance Control Planes in Autonomous Systems*. Zenodo. DOI: https://doi.org/10.5281/zenodo.19241732

Willis, J. (2026c). *Runtime Governance Architecture: Consistent Governance Execution for Enterprise Systems and Autonomous Agents*. Sustainable Future Tech Publishing. DOI: https://doi.org/10.5281/zenodo.19286845

Willis, J. M. (2026d). *The PBSAI Governance Ecosystem: A Multi-Agent AI Reference Architecture for Securing Enterprise AI Estates*. arXiv. Available at: https://arxiv.org/pdf/2602.11301

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. Available at: https://arxiv.org/abs/2210.03629